

FOR SOFTWARE ENGINEERS: THE CYCLE, ON REAL MONEY

TDD For Java

The step everyone skips

A test that has never failed has never proved anything. It has only agreed with you.

You already write tests. You write them after the code, they pass on the first run, and that feels like the system working.

This sheet is the cycle on a refund rule rather than a calculator, the evidence with both halves attached, and the objection a sceptic raises at minute three answered honestly, including the part where they are right.

Every figure here was read at the primary source in September 2026 and is quoted with its limitations.

In scope

- The red step that proves something, and the one that proves nothing
- Writing the refusals rather than the happy path
- Invariants that depend on history rather than on the request
- What the evidence says, including the cost
- The honest difference between test-first and test-after

Out of scope

- Allegiance to TDD doctrine. Four rules are given up on page one
- Mocking frameworks, which are a different argument
- Coverage targets. Coverage says what ran, not what was checked
- Any claim that an agent writing your tests is the same as a specification

If you read one section, read the one on what a regression run actually tells you. That is where the method stops being a ritual.

01 What the evidence says, both halves, checked September 2026

Meta-analysis, 27 studies, external quality	small positive effect
Meta-analysis, 27 studies, productivity	little to no discernible effect
Industrial subgroup	larger quality gain AND larger productivity drop
4 teams, 3 Microsoft + 1 IBM, defect density	40% to 90% lower
The same four teams, time to complete	15% to 35% longer
What that licenses	real, modest, and it costs you time
What it does NOT license	"TDD reduces defects by X percent"

The four industrial teams are observational case studies, not a controlled experiment, and the cost sits in the same paper as the benefit.

03 Four things this sheet will not defend

"EVERY PRODUCTION LINE NEEDS A FAILING TEST FIRST." Too rigid. Infrastructure wiring, spikes, generated code and trivial mappings are poor candidates and everybody knows it.

"THE CYCLES MUST ALWAYS BE TINY." Small feedback loops are useful. Writing the smallest imaginable assertion is not virtuous by itself.

"ONE HUNDRED PERCENT COVERAGE." Coverage tells you what executed, not whether meaningful behaviour was verified. Mutation testing exists partly because covered code can still be badly tested.

"ONE ASSERTION PER TEST." One BEHAVIOUR per test is the better heuristic, and one behaviour can legitimately need several assertions.

Pretending the method has no cost is how it lost the room. Naming the cost first is what earns the rest of the page.

`Editorial judgement, stated as such`

04 Write the refusals. More of them than feels reasonable

Seven tests against a refund rule. ONE says what must be allowed. SIX say what must be refused.

That ratio is the most transferable thing here. The happy path is already in your head and you do not need a test to remember it. The refusals are the cases that become incidents, and they are the ones that will not occur to you once you are looking at working code.

AND THREE OF THEM DO NOT LOOK AT THE REQUEST AT ALL. A refund is refused if it exceeds what is LEFT rather than what was captured. A FAILED attempt consumes none of the balance, so a later refund still goes through. The same idempotency key refunds once. Those describe what the balance MEANS given a history. That is the domain, not validation.

A suite that only proves the happy path is a suite that agrees with whoever wrote the code.

`Measured on the machine that built this sheet, not quoted`



05 Red, for the right reason

Run the suite BEFORE the implementation exists, against a skeleton that throws "not implemented yet". Seven run, four fail, three error.

Now look at WHY they failed. Not "class not found". Not "did not compile". Every test compiled, ran, reached the method under test, and failed on the thing you put there deliberately.

THAT IS WHAT THE RED STEP IS FOR. It proves each test runs, reaches the code, and is checking something. A red step that only proves you have not written the class yet has proved nothing about your test, and skipping this is what turns the cycle into a ritual.

Seven tests, four failures and three errors, every one of them failing on a deliberate `UnsupportedOperationException` rather than on a missing class.

Measured on the machine that built this sheet, not quoted

06 The run that makes the case

One plausible simplification, exactly the kind that passes review. The refundable balance is computed from the refund history, somebody decides that is a needless indirection, and compares against what was CAPTURED instead.

SIX TESTS STAY GREEN. One does not, and it names the case and the line: refuses a second refund beyond what is left. Ten thousand captured, six thousand already refunded, and a five thousand refund goes through.

SIT WITH THE SHAPE OF THAT RESULT, because the shape is the argument. Six tests agreed with the change. They were not wrong, they were not looking at that. One disagreed, by name, in 41 milliseconds, before review and before a customer.

THAT IS WHAT "THE TESTS ARE A SPECIFICATION" MEANS. Not that they document the code. That when somebody changes the code, the specification is the thing that objects.

Six of seven tests pass under the regression. The failure names `refusesSecondRefundBeyondRemaining` and the line number.

Measured on the machine that built this sheet, not quoted

07 The objection at minute three, answered honestly

It is not "this is slow". It is: why does it matter whether I write the exact same good test five minutes before the code or five minutes after it?

THE DISHONEST ANSWER is "because that is the discipline". It loses, and it deserves to.

THE HONEST ONE: sometimes it genuinely does not matter. If you understand the requirement, implement it correctly, and then independently write a suite capable of catching realistic faults, test-after gives you the same confidence. Test-after tests are not inherently inferior.

The advantage is narrower and it is about YOU. Once you have implemented it, you know how it works, and that knowledge changes which tests occur to you. You will naturally confirm the structure you just built. Test-first asks a different question: before I know how this will work, what behaviour must be true?

The advantage of test-first is not test quality in the abstract. It is that after implementing, you are no longer a neutral party.

Measured on the machine that built this sheet, not quoted

08 The AI argument, and why the obvious version is wrong

THE OBVIOUS VERSION: an agent implements in nine seconds, so the test is what is left, so test first matters more. That does not follow.

THE OBJECTION THAT KILLS IT: if the same tool writes your test AND your implementation from one prompt, you do not have an independent specification. You have the same reasoner marking its own exam, and ten green tests do not help if all ten encode the same misunderstanding.

WHAT SURVIVES, and it is the useful version: this does not make test driven development more valuable. It makes INDEPENDENT SPECIFICATION more valuable, and writing the test first is one way of forcing that separation.

Generating code got cheaper much faster than deciding what the code should do. Nobody can tell you whether a second refund of fifty after a sixty refund of a hundred capture should fail. That is not a coding question.

Never let the system that produced the work be your only source of evidence that the work is correct.

The same principle as the verification episode on this channel

09 When not to bother

A SPIKE, where you are writing code to find out what you are building. Specifying the answer before you have found it is theatre.

A TRIVIAL MAPPING, or generated code, where the test restates the code in a second language.

WIRING ONE FRAMEWORK TO ANOTHER, where you end up testing somebody else's library badly.

AND WHEN YOU CANNOT SAY what observable behaviour you expect. At that point you do not have a missing test. You have a design question you have not finished.

Where the expected behaviour is obvious to everyone, the test is documentation rather than a discovery.

Operational practice, not vendor documentation

Before you call a suite a specification

1. How many of these tests say what must be REFUSED?
2. Did I watch it fail before I made it pass?
3. Did it fail for the right reason, or just because the class was missing?
4. Is there an invariant here that depends on history rather than on the request?
5. If somebody makes a plausible simplification, which test objects?
6. Would that test name the case, or just go red?
7. Were these tests written by the same thing that wrote the implementation?
8. What does this suite cost us on every change, and is it worth that?
9. Which of these tests has never failed, not even once?
10. What behaviour is true in production that no test here describes?



Everything else

Site

<https://code-with-sam-dev.github.io>

YouTube

<https://www.youtube.com/@CodewithSam-Dev>

LinkedIn

<https://www.linkedin.com/company/code-with-sam-dev>

X

<https://x.com/CodeWithSamDev>

TikTok

<https://www.tiktok.com/@codewithsamdev>

Instagram

<https://www.instagram.com/codewithsamdev>

GitHub

<https://github.com/code-with-sam-dev>

If this was useful, a like and a subscribe help more than you would think. And tell me in the comments which of these you have actually shipped.

Java and OpenJDK are trademarks or registered trademarks of Oracle and/or its affiliates. Eclipse Temurin is a trademark of the Eclipse Foundation. They appear here only to identify the technology being discussed. Nothing here implies endorsement by, or affiliation with, any of them.

Technical claims verified 2026-09-15 against the documentation named beside each one. Vendors change behaviour and defaults between versions, so check the version you are actually running before you rely on any of it.